

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA
MACROAREA DI SCIENZE MATEMATICHE, FISICHE E NATURALI



CORSO DI STUDIO IN
INFORMATICA

TESI DI LAUREA IN
INFORMATICA

TITOLO

VulcaTest: Framework agentico per verifica di conformità
e self-healing di scenari Cyber Range

Relatore:

Chiar.mo Prof.
Francesco Pasquale

Laureando:

matricola: 0342634
Luca Gugliotta

Anno Accademico 2025/2026

Indice

1	Introduzione	2
1.1	Il problema: verificare macchine vulnerabili generate automaticamente	2
1.2	Obiettivi e contributi della tesi	2
1.3	Struttura della tesi	2
2	Contesto	3
2.1	Cyber Range, CTF e macchine Boot-to-Root	3
2.2	Intended path e conformità di una macchina didattica	3
2.3	L’ecosistema VulcAIn: VulcaMind, VulcaForge e VulcaShip	4
2.4	Agenti basati su LLM: ReAct e Plan-and-Execute	5
2.5	Harness agentici	6
3	Architettura e implementazione di VulcaTest	8
3.1	Requisiti e limiti delle soluzioni generiche	8
3.2	Principi di progettazione del sistema	9
3.3	Architettura generale e flusso di coordinamento	10
3.4	Il Planner: generazione e formalizzazione dell’Attack Plan	11
3.5	L’Orchestrator: controllo del workflow e gestione dello stato	14
3.6	L’Executor: esecuzione e verifica degli step	14
3.7	Il Bridge di esecuzione: gestione dei tool e delle interazioni con il target	16
3.8	Il Final Evaluator: valutazione finale e Root Cause Analysis	18
3.9	Prompt engineering e definizione dei ruoli agentici	18
3.10	Modello locale e configurazione dei parametri	19
3.10.1	Modello e runtime	19
3.10.2	Dimensionamento della finestra di contesto	20
4	Il self-healing closed-loop in VulcaTest	22
4.1	Il nodo di healing nel workflow closed-loop	22
4.2	Il ticket di healing	22
4.3	L’agente di healing: delega a un harness agentico	23
4.4	Prompt dell’agente di healing e perimetro di modifica	24
4.5	Tracciamento e validazione delle modifiche	25
4.6	Chiusura del ciclo: rebuild dell’ambiente e regression testing	26
5	Valutazione sperimentale	28
5.1	Obiettivi della valutazione e Test Oracle Problem	28
5.2	Metriche di valutazione	28
5.3	Dataset: macchine di riferimento e perturbazioni controllate	28
5.4	Configurazione sperimentale	28

5.5	Risultati	28
5.5.1	Riconoscimento, diagnosi e riparazione	28
5.5.2	Confronto tra modelli	28
5.5.3	Studio di ablazione dei ruoli	28
5.6	Casi di studio	28
5.6.1	Pizzeria_B2R: validazione closed-loop end-to-end	28
5.7	Discussione e limiti della valutazione	28
6	Conclusioni e sviluppi futuri	29
6.1	Sintesi dei contributi	29
6.2	Limiti della soluzione attuale	29
6.3	Sviluppi futuri	29
	Riferimenti bibliografici	30
	Ringraziamenti	33

Sommario

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1 Introduzione

- 1.1 Il problema: verificare macchine vulnerabili generate automaticamente
- 1.2 Obiettivi e contributi della tesi
- 1.3 Struttura della tesi

2 Contesto

LUI?

VulcaTest verifica macchine didattiche generate automaticamente. Per descriverlo servono alcuni concetti legati al suo contesto, introdotti in questo capitolo.

2.1 Cyber Range, CTF e macchine Boot-to-Root

Un Cyber Range è una piattaforma interattiva che simula reti, sistemi, strumenti e applicazioni, usata per l'addestramento e la sperimentazione nel campo della sicurezza informatica [1, 2]. Il vantaggio è ^{QUELLO DI AVERE} che si ha un ambiente isolato, in cui gli studenti possono attaccare i sistemi senza conseguenze reali ~~e lo scenario può essere ripristinato~~ ^{CON UNO RIPRISTINABILE} più volte.

In ambito didattico i Cyber Range ospitano spesso sfide nel formato Capture The Flag (CTF), in cui il partecipante deve sfruttare una o più vulnerabilità per recuperare una flag, spesso una stringa segreta che dimostra il superamento della sfida.

Tra le CTF usate in ambito didattico sono molto diffuse le macchine Boot-to-Root (B2R), sfide costruite attorno a un singolo sistema. Il partecipante parte senza alcun accesso, ottiene una shell come semplice utente e da lì arriva ai privilegi di amministratore (root). Ogni passaggio di livello richiede tre attività: l'*enumeration*, cioè la raccolta di informazioni sul sistema target, l'individuazione dei vettori di attacco, cioè di vulnerabilità o misconfigurazioni sfruttabili, e il loro *exploitation*. ^{* SPIEGARE} Il passaggio finale da utente a root prende il nome di *privilege escalation*. Le macchine generate da VulcaTest appartengono a questa categoria.

2.2 Intended path e conformità di una macchina didattica

Le vulnerabilità di una macchina B2R didattica sono scelte e concatenate dall'autore, in modo che ogni fase corrisponda a un obiettivo formativo. La sequenza di passaggi progettata prende il nome di *intended path* e di norma è documentata in un writeup. ^{LA SOLUZIONE PREVISTA NON E' DICENDO L'UNICA} Non è necessariamente l'unica soluzione, perché in questo genere di challenge la scoperta di percorsi alternativi è spesso incoraggiata, ma resta il percorso su cui l'autore ha costruito gli obiettivi didattici, ed è quello su cui si concentra questa tesi.

Da questa caratteristica deriva il requisito principale di una macchina didattica: deve poter essere risolta lungo l'intended path. In questa tesi una macchina si dice conforme quando ogni passaggio del percorso progettato funziona sulla macchina in esecuzione. Raggiungere la root non basta a dimostrarlo: se un passaggio non funziona, il partecipante può comunque arrivare alla root per un'altra strada. ^{BRUTTO} In questo caso la macchina è **compromettibile** ma non conforme, e non insegna ciò per cui è stata progettata. <sup>TROPPO SCONE SSE
CI SI RIFERISCE AD UNA MACCHINA CONFORME</sup>

Realizzare macchine di questo tipo richiede molto tempo e competenze. Oltre alla progettazione, bisogna configurare il sistema operativo, i servizi, gli utenti e i permessi in modo che le vulnerabilità volute siano sfruttabili senza introdurne altre. La verifica consiste infine nel risolvere la macchina seguendo il writeup, e va ripetuta a ogni modifica, perché anche un piccolo cambiamento di configurazione può interrompere un passaggio del percorso.

PERFOI
PROCEDE
CON UN'A

INVALIDARE?

2.3 L'ecosistema VulcaIn: VulcaMind, VulcaForge e VulcaShip

VulcaIn nasce per ridurre il lavoro richiesto dalla realizzazione e lasciare all'autore più spazio per la progettazione. È un ecosistema modulare, ideato e sviluppato da Danilo Dell'Orco e Michele Salvatori, che combina agenti AI e Infrastructure as Code (IaC)¹. La generazione è divisa in tre moduli (Figura 1):

- **VulcaMind**, a partire da una descrizione iniziale, definisce struttura, storyline, percorso di attacco e soluzione della challenge. La storyline e il writeup che produce descrivono l'intended path: la prima l'ordine dei passaggi, il secondo i dettagli tecnici per eseguirli.
- **VulcaForge** traduce la progettazione in infrastruttura. Un agente compone i moduli di un registro di vulnerabilità, servizi e configurazioni in una *ricetta IaC*, un file YAML dichiarativo che descrive l'intera macchina. Dalla ricetta vengono poi generati il playbook Ansible [3] e i file di supporto, come i Dockerfile [4] delle applicazioni web.
- **VulcaShip**, gestisce il deploy della macchina virtuale sull'infrastruttura di virtualizzazione.

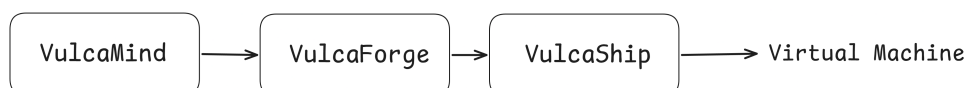


Figura 1: Pipeline VulcaIn originale, senza VulcaTest.

La pipeline originale è lineare e priva di retroazione. Se l'agente di VulcaForge commette un errore, ad esempio dimentica di esporre una porta, la macchina viene messa in esecuzione già difettosa. I controlli presenti verificano che i servizi siano avviati, non che la macchina sia risolvibile lungo l'intended path, e il problema emerge solo quando qualcuno la prova manualmente e la corregge a mano. Questa tesi propone VulcaTest (Capitoli 3 e 4), progettato per inserirsi dopo VulcaShip e chiudere il ciclo: verifica la conformità della macchina in esecuzione con un penetration test agentico e, se trova una non conformità, la ripara con il nodo di healing. La

IN QUESTA PIPELINE SOLO

I PROBLEMI

SONO

DAVE

ALI

¹Infrastructure as Code: approccio in cui l'infrastruttura (sistemi, servizi, configurazioni) viene descritta in file di testo eseguibili e versionabili, invece di essere configurata a mano.

UNIFORMARE LE 2 COSE + SCORREVOLE

fase di test non dipende dal resto della pipeline: } le servono la documentazione della challenge e la macchina eseguita come container Docker, e può quindi verificare anche macchine prodotte al di fuori di VulcAIn. Il nodo di healing invece lavora sulla ricetta IaC e usa il generatore di VulcaForge per ricostruire la macchina, quindi richiede VulcaForge. VulcaTest è pensato per integrarsi nel workflow di VulcAIn, e la sua integrazione automatica è uno degli sviluppi futuri.

BRUTTA
TROPPO
DIRETTA

LASCIA QUESTO
TOGLI QUELLO
PRIMA

2.4 Agenti basati su LLM: ReAct e Plan-and-Execute

VulcaMind, VulcaForge e VulcaTest affidano parte del proprio lavoro ad agenti basati su Large Language Model (LLM). Un LLM è un modello addestrato su grandi quantità di testo che, ricevuto un input testuale (prompt), ne genera una continuazione, ma da solo non può osservare un ambiente né agire su di esso. Si parla di *language agent*, o agente basato su LLM, quando il modello è inserito in un'architettura che gli permette di interagire con un ambiente esterno attraverso memoria, ragionamento, pianificazione e azioni [5, 6].

Per descrivere la struttura di un agente di questo tipo si può usare il framework CoALA (*Cognitive Architectures for Language Agents*), che lo scompone in tre elementi: la memoria, le azioni che può compiere e il ciclo decisionale con cui sceglie, a ogni passo, quale azione eseguire [5]. Le azioni possono essere interne, come il ragionamento, che modifica solo ciò che l'agente sa, oppure esterne, che agiscono sull'ambiente e prendono il nome di *grounding*. Lanciare un comando su una macchina remota è un esempio di azione esterna.

Il ciclo decisionale può essere organizzato in modi diversi, e due schemi molto diffusi sono ReAct e Plan-and-Execute. In ReAct (*Reasoning + Acting*) [7] l'agente alterna un ragionamento sullo stato corrente, un'azione sull'ambiente e l'osservazione del risultato, che guida il ragionamento successivo. Il ragionamento esplicito, introdotto con il Chain-of-Thought [8], aiuta a scomporre un problema ma, se resta isolato dall'ambiente, può propagare errori e allucinazioni. Un agente che agisce senza ragionare tende invece a perdere di vista l'obiettivo e a ripetere azioni inutili. ReAct unisce i due aspetti, ma ha anche dei limiti: l'agente può entrare in cicli in cui ripete la stessa azione, e la sequenza di ragionamenti e osservazioni può riempire la finestra di contesto².

INTEGRA
MEGLIO

Il Plan-and-Execute segue un approccio diverso: l'agente costruisce prima un piano completo, scomposto in passi, e solo dopo li esegue uno alla volta. Nella classificazione di Wang et al. si tratta di una pianificazione senza feedback, mentre ReAct è l'esempio tipico di pianificazione con feedback [6]. Il vantaggio è un'esecuzione più prevedibile,

²Quantità massima di testo, misurata in token, che il modello può considerare in una singola chiamata.

perché i passi sono noti in anticipo. Il limite è che il piano non cambia anche quando l'ambiente restituisce qualcosa di inatteso.

Oltre al ciclo del singolo agente, conta il modo in cui il lavoro viene distribuito quando i ruoli sono più di uno. MetaGPT mostra che la collaborazione tra agenti migliora quando ogni ruolo ha compiti definiti e passa al successivo artefatti strutturati, invece di dialogare liberamente [9].

2.5 Harness agentici

Il comportamento di un agente non dipende solo dal modello, ma anche dal software che lo circonda, chiamato *harness*. L'harness costruisce il prompt, espone i tool ed esegue le chiamate, gestisce il ciclo di esecuzione e decide quando interromperlo. Si occupa anche del contesto e dei permessi, cioè di cosa l'agente può leggere, modificare ed eseguire. Il suo peso sui risultati non è trascurabile: in Cybench, a parità di task, lo stesso cambio di harness migliora le prestazioni di un modello e peggiora quelle di un altro [10].

Wang et al. distinguono due modi per migliorare un agente senza modificare i pesi del modello: il *prompt engineering*, che agisce sulle istruzioni, e il *mechanism engineering*, che agisce sui meccanismi costruiti attorno al modello [6]. È in questa seconda categoria che si colloca il lavoro di questa tesi.

↑ NON SOLO
PURE 1

Gli harness più diffusi sono general-purpose, come Google Antigravity [11] e Claude Code [12], e sono pensati per compiti aperti come lo sviluppo software: l'agente ragiona sul codice e opera attraverso editor e terminale. Tramite il Model Context Protocol (MCP)³ [13] possono essere estesi con nuovi tool, e quindi collegati anche a strumenti di sicurezza offensiva come quelli raccolti da HexStrike AI [14].

Accanto a questi esistono harness pensati appositamente per il penetration testing. PentestGPT nasce dall'osservazione che, nei test lunghi, gli LLM tendono a perdere il contesto della sessione e a dimenticare i risultati ottenuti nei passi precedenti. Per questo divide il lavoro tra tre moduli: uno di ragionamento, che tiene traccia dello stato complessivo del test, uno che traduce i passi in comandi e uno che riassume gli output degli strumenti [15]. Cybench affronta invece il problema della valutazione: misura le capacità degli agenti su challenge CTF professionali e, scomponendo ogni challenge in subtask intermedi, mostra fin dove arriva l'agente anche quando non ottiene la flag [10]. In entrambi i casi l'obiettivo è risolvere la challenge, o misurare quanto bene il modello riesce a farlo. Nessuno dei due verifica che una macchina sia risolvibile lungo un percorso dato.

³Protocollo aperto che standardizza il modo in cui un'applicazione basata su LLM si collega a strumenti e fonti di dati esterne.

In tutti questi casi l'harness lascia all'agente ampia autonomia nel raggiungere l'obiettivo. La Sezione 3.1 mostra perché, nella verifica della conformità, questa autonomia diventa un limite.

3 Architettura e implementazione di VulcaTest

Le macchine generate da VulcAIIn ~~devono essere verificate prima di essere usate.~~

- Non basta che vengano costruite senza errori: devono anche poter essere risolte lungo il percorso didattico previsto. VulcaTest svolge questa verifica attraverso un penetration test agentico, in cui ogni passaggio viene confermato da evidenze raccolte sul target. La correzione delle non conformità è descritta nel Capitolo 4.

3.1 Requisiti e limiti delle soluzioni generiche

L'obiettivo di VulcaTest è verificare che una macchina didattica vulnerabile sia conforme, nel senso definito nella Sezione 2.2, e in caso contrario correggerla.

Una prima sperimentazione è stata condotta con Google Antigravity ^{WISITE?} e il modello Gemini 3.8 Flash High, configurato con un system prompt dedicato e collegato ai tool necessari tramite il protocollo MCP [11]. Lo scopo di questa sperimentazione era capire se un modello di frontiera, dotato di tool e istruzioni adeguate, fosse sufficiente a condurre in autonomia il penetration test. Le prove hanno mostrato alcuni limiti. I primi dipendono dai modelli di frontiera a cui gli harness commerciali sono legati:

- RIPETI
- Il primo limite riguarda i guardrail dei modelli di frontiera. Alcune operazioni tipiche del penetration testing, anche se eseguite in un ambiente didattico controllato, possono essere bloccate dai filtri di sicurezza del provider e interrompere il testing.
 - Il secondo limite è il costo. Lo sviluppo e la valutazione del sistema richiedono molte esecuzioni complete, ciascuna composta da numerosi turni e chiamate ai tool. Con un modello commerciale il costo cresce con il numero e la durata dei test.

Entrambi i problemi si riducono notevolmente eseguendo il modello in locale. La scelta del modello e la sua configurazione sono descritte nella Sezione 3.10.

Anche eliminando i limiti derivanti dalla scelta del modello, resta un problema più generale. Gli harness general-purpose sono pensati per lasciare all'agente la massima autonomia nel raggiungere l'obiettivo. Per VulcaTest, però, il risultato finale non basta: bisogna verificare se la macchina è stata risolta lungo il percorso previsto e se ogni passaggio è supportato da evidenze osservabili.

Durante le prove di queste soluzioni generiche sono emerse tre criticità:

1. **Eccessiva libertà operativa.** L'agente tende a privilegiare il raggiungimento dell'obiettivo rispetto all'aderenza all'*intended path*. Sono stati osservati tentativi di usare canali esterni alla challenge, come l'accesso diretto al demone Docker dell'host o l'ispezione del filesystem sottostante. In questi casi il test dimostra soltanto che la macchina è compromettibile, non che lo sia attraverso le sole vulnerabilità didattiche previste.
2. **Auto-certificazione del risultato.** Lo stesso agente che esegue un passaggio dell'attacco ne valuta anche l'esito. Una fase può così essere considerata completata sulla base di una deduzione o di un output parziale, senza una prova esplicita per ogni condizione richiesta. Nel conformance testing questo porta ad accettare macchine in cui uno dei passaggi in realtà non funziona.
3. **Assenza di controllo sull'esecuzione.** In un harness generico, una volta inviato il prompt, il ciclo agentico procede in autonomia e non è possibile intervenire sulle singole fasi: stabilire quando fermare l'agente, quali tool rendergli disponibili in un dato momento o quali dati registrare. Con un orchestratore sviluppato appositamente, invece, ogni fase e ogni operazione possono essere controllate, ed è possibile raccogliere le metriche necessarie alla valutazione del sistema.

Un caso concreto è emerso durante il test di *Pizzeria_B2R*, una delle macchine generate dalla pipeline. Il percorso previsto richiedeva di scoprire un endpoint nascosto attraverso una chat di assistenza. A causa di un difetto di generazione la chat non funzionava, ma il collegamento all'endpoint era rimasto visibile nel codice HTML della pagina. L'agente ha seguito il collegamento diretto, ha proseguito fino a ottenere una shell e ha considerato il test superato. La macchina risultava quindi compromettibile, ma non conforme: uno dei passaggi del percorso didattico non funzionava.

Questo caso mostra che risolvere una challenge non equivale a verificarne la conformità. Da qui nasce VulcaTest: un'architettura in cui l'agente opera entro un perimetro controllato e l'esito del test dipende dalle evidenze raccolte durante l'esecuzione, non dalla sola valutazione del modello.

3.2 Principi di progettazione del sistema

Prima di progettare l'architettura di VulcaTest sono stati fissati cinque principi, utilizzati come vincoli da rispettare. Non descrivono ancora componenti concrete, ma le proprietà che il framework deve garantire.

1. **Deterministico quando possibile, probabilistico quando necessario.** Non tutte le operazioni del workflow richiedono un modello generativo. Ciò che può essere espresso con regole precise viene affidato a codice deterministico. Il

E' CORRETTO?

modello interviene solo dove servono interpretazione, adattamento al contesto o la scelta tra più azioni possibili.

2. **Esecuzione basata su evidenze.** L'esito di uno step non deve dipendere soltanto dal giudizio della componente che lo ha eseguito. Ogni risultato deve essere accompagnato dalle evidenze raccolte sul target, in modo da poter ricostruire cosa è stato fatto e su quali elementi si basa la valutazione.
3. **Separazione delle responsabilità.** Pianificazione, coordinamento, esecuzione e valutazione sono assegnate a componenti distinte, ciascuna con un ruolo e compiti definiti (impostazione *role-based*).
4. **Modularità.** Le componenti devono restare il più possibile indipendenti tra loro. Deve essere possibile cambiare modello, parametri di inferenza o backend senza modificare la struttura del workflow, e aggiungere nuovi tool senza toccare le parti che non li usano.
5. **Pianificazione globale, ReAct nell'esecuzione.** Il sistema deve lavorare su due livelli. A livello globale segue un approccio Plan-and-Execute (Sezione 2.4): il percorso viene definito prima dell'esecuzione e non cambia durante il test, perché quello da verificare è uno solo: il percorso progettato dall'autore. All'interno di ogni step, l'esecuzione segue invece un ciclo ReAct: l'azione successiva viene decisa in base agli output osservati sul target.

3.3 Architettura generale e flusso di coordinamento

VulcaTest è composto da quattro componenti principali (Planner, Orchestrator, Executor e Final Evaluator), affiancate dal Bridge, che gestisce l'accesso ai tool, e dal nodo di healing, che verrà descritto nel Capitolo 4.

Il workflow di VulcaTest procede in modo lineare. Il Planner legge la documentazione della challenge e genera l'Attack Plan, cioè la sequenza di step da verificare. Il piano passa poi all'Executor, che lo esegue uno step alla volta: per ogni step un LLM sceglie le azioni da compiere, che vengono eseguite attraverso il Bridge sui tool di una macchina Kali dedicata. La Kali fornisce la postazione di attacco, mentre la macchina target da verificare è raggiunta attraverso il proprio indirizzo IP. Al termine del piano, oppure al primo step fallito, il Final Evaluator analizza i risultati raccolti e produce la valutazione finale della run. In caso di non conformità, il controllo può passare al nodo di healing. L'avanzamento tra queste fasi è governato dall'Orchestrator, che decide quale componente eseguire e quando passare alla successiva, senza lasciare questa scelta all'LLM.

Prima di descrivere l'architettura nel dettaglio, è utile inquadrare il tipo di agente che VulcaTest realizza. Facendo riferimento al modello CoALA (Sezione 2.4), VulcaTest può essere descritto come un agente modulare: funzioni diverse sono separate in

componenti specializzate, che restano però coordinate all'interno di un unico flusso di esecuzione e condividono una working memory. Questa memoria raccoglie le informazioni necessarie al compito corrente e viene aggiornata durante l'interazione dell'agente con l'ambiente.

→ In VulcaTest questo ruolo è svolto da `VulcaTestState`, la struttura dati condivisa che mantiene lo stato della run e che viene letta e aggiornata dalle diverse componenti.

→ Le componenti non si scambiano informazioni sotto forma di testo libero, ma attraverso strutture dati definite, che il codice può controllare prima di passarle alla fase successiva: ogni step del piano è rappresentato da un oggetto `TestStep` e il suo esito da uno `StepResult`. Grazie allo stato condiviso, ogni componente trova le informazioni di cui ha bisogno senza doverle ricostruire dal testo generato dall'LLM.

VulcaTest può essere avviato in due modalità:

- **Execution Mode:** se è già disponibile un Attack Plan generato e validato (`ATTACK_PLAN.md`), la pianificazione viene saltata e il test parte direttamente dall'esecuzione.
- **Planning & Execution Mode:** se il piano non esiste, o se ne viene richiesta la rigenerazione, viene eseguito prima il Planner. Il piano prodotto viene validato e poi passato allo stesso flusso di esecuzione.

In entrambi i casi, una volta disponibile l'Attack Plan, il test prosegue attraverso lo stesso workflow.

3.4 Il Planner: generazione e formalizzazione dell'Attack Plan

Il Planner trasforma la documentazione della challenge in un Attack Plan (`ATTACK_PLAN.md`), suddiviso in fasi. Per ogni fase, l'Attack Plan indica cosa deve essere raggiunto e quali condizioni vanno verificate prima di proseguire. Nel seguito i termini *fase* e *step* indicano la stessa unità, che nel codice corrisponde a un oggetto `TestStep`, mentre le singole azioni svolte dall'Executor all'interno di uno step sono chiamate *turni*.

Il Planner riceve tre documenti: `DESCRIPTION.md`, scritto dall'utente, e `STORYLINE.md` e `WRITEUP.md`, generati da VulcaMind. Poiché hanno ruoli diversi, i tre documenti possono contenere informazioni non del tutto allineate. Per evitare che sia il modello a risolvere i conflitti in modo arbitrario, il prompt stabilisce un ordine di priorità:

- `STORYLINE.md` definisce il percorso previsto e l'ordine delle fasi.

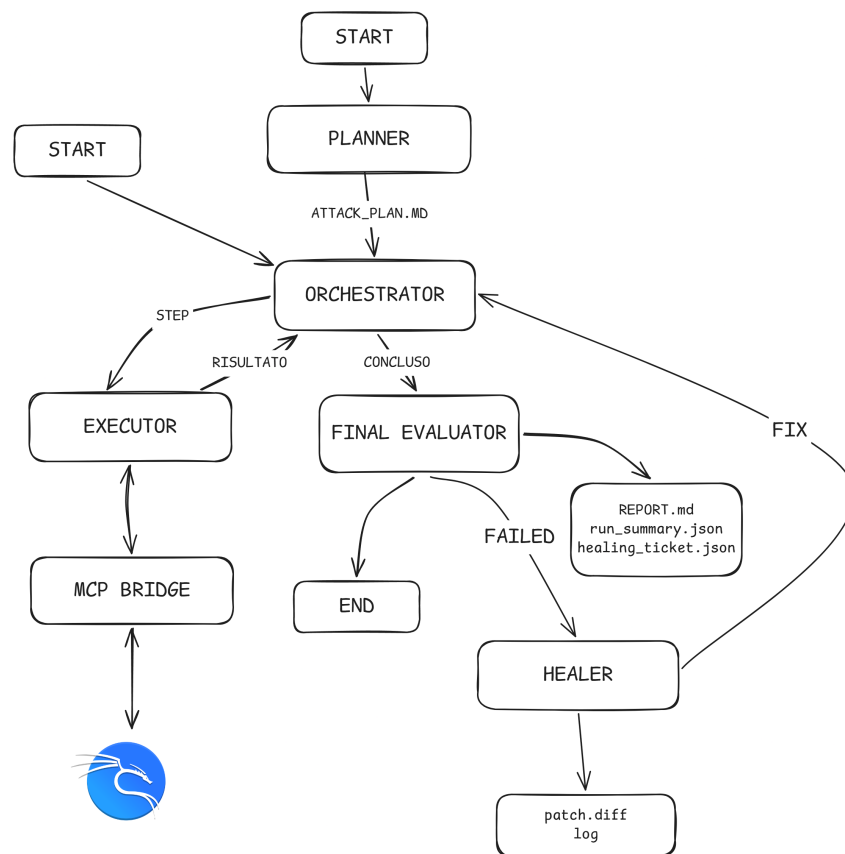


Figura 2: Architettura e flusso di coordinamento di VulcaTest: transizioni tra i nodi dello StateGraph gestite dall'Orchestrator.

- `WRITEUP.md` fornisce i dettagli tecnici per eseguirle.
- `DESCRIPTION.md` contiene le informazioni generali sulla challenge.

In caso di conflitto tra le tre fonti il Planner mantiene il percorso della Storyline, usa il Writeup come riferimento tecnico e non introduce passaggi assenti dai documenti.

Anche porte, credenziali, percorsi e altri parametri devono provenire dai documenti di input. Se un valore non è ancora noto viene usato un placeholder esplicito, come `<TARGET_IP>`, che viene risolto durante l'esecuzione.

Il Planner lavora in due stadi. Nel primo, un LLM legge i documenti della challenge e genera `ATTACK_PLAN.md`, un documento Markdown in cui le informazioni strutturate (liste, parametri, comandi) sono racchiuse in blocchi YAML. Il formato YAML è stato preferito al JSON perché ha una sintassi più permissiva.

IN QUESTO SPECIFICO CASO D'USO

Nel secondo stadio, un parser in Python verifica la struttura del documento e converte ogni fase in un oggetto `TestStep`. Il modello resta responsabile dell'interpretazione dei documenti, mentre il codice controlla che il risultato sia utilizzabile dal resto del workflow. Se il piano ha sezioni mancanti o non rispetta il formato, viene rifiutato prima dell'avvio del test, in modo che un errore di generazione non arrivi alle fasi successive.

Ogni `TestStep` contiene un identificativo, l'obiettivo della fase, i tool consentiti, i valori che la fase deve produrre e una checklist di condizioni da verificare. Al termine dello step è la checklist a stabilire se lo step è stato completato. Per questo la sua costruzione segue alcune regole precise, definite nel system prompt del Planner:

- **Tutti i requisiti sono obbligatori.** Le voci della checklist sono in AND e non possono essere marcate come opzionali. Un controllo che non è un requisito rigido della fase va rimosso o spostato nella fase in cui lo diventa.
- **Il meccanismo di scoperta previsto va verificato.** Se una risorsa deve essere ottenuta attraverso uno specifico passaggio, la checklist richiede di verificare quel passaggio. Raggiungere la stessa risorsa con una scorciatoia non è sufficiente.
- **Ogni controllo si basa su un risultato osservabile.** Deve trattarsi di un output concreto, come una risposta HTTP, una porta aperta, un file presente o un valore estratto, ottenibile con i privilegi disponibili in quella fase.
- **Anche un fallimento può essere il comportamento atteso.** Se uno step prevede intenzionalmente un errore o un rifiuto, la checklist verifica che quel comportamento si presenti, invece di richiedere l'esito positivo del comando.

3.5 L'Orchestrator: controllo del workflow e gestione dello stato

L'Orchestrator controlla l'avanzamento dell'intero test, compresa la fase di healing. Con questo termine non si indica un singolo modulo, ma l'insieme della logica che coordina il workflow: quale componente eseguire, in quale ordine e quando il test deve proseguire o terminare. L'Orchestrator non decide come completare uno step, ma soltanto cosa deve succedere dopo.

Questa logica è implementata con LangGraph [16], che rappresenta il workflow come un grafo a stati. I nodi del grafo sono **orchestrator**, **executor**, **final_evaluator** e **healer** (Figura 2), mentre gli archi definiscono le condizioni con cui il controllo passa da un nodo all'altro. Le transizioni sono decise interamente dal codice.

I nodi condividono le informazioni della run attraverso **VulcaTestState**, introdotto nella Sezione 3.3. LangGraph passa lo stato a ogni nodo, che al termine della propria esecuzione restituisce gli aggiornamenti prodotti. **VulcaTestState** contiene la sequenza dei **TestStep**, l'indice dello step corrente, lo stato generale della run, i risultati degli step già eseguiti, i valori verificati e le sessioni terminali ancora attive, oltre ai dati di ~~telemetria~~ usati per i report finali.

METRICHE?

Tra i nodi del grafo ce n'è uno chiamato anch'esso **orchestrator**, da non confondere con l'Orchestrator nel suo insieme. Si tratta di un nodo di smistamento: seleziona il **TestStep** corrente e, se il piano è terminato, passa il controllo al Final Evaluator, altrimenti avvia l'Executor.

Se lo step ha successo, lo stato viene aggiornato e il controllo torna al nodo **orchestrator**, che seleziona lo step successivo. Se invece fallisce, il test si interrompe secondo una logica *fail-fast* e il controllo passa direttamente al Final Evaluator.

Dopo la valutazione finale, una run completata termina. Se invece è stata rilevata una non conformità, il self-healing è abilitato e restano tentativi di correzione disponibili, il controllo passa al nodo **healer**. Al termine della correzione il test riparte dall'inizio.

3.6 L'Executor: esecuzione e verifica degli step

L'Executor è la componente Python che gestisce l'esecuzione dei singoli **TestStep**. Per ogni step prepara il contesto, mette a disposizione i tool consentiti e invoca l'LLM, che agisce come agente e decide di volta in volta quale azione eseguire.

L'interazione segue un ciclo ReAct (Sezione 2.4): a ogni turno l'agente ragiona sullo stato corrente, sceglie un'azione, ne osserva il risultato e decide come proseguire.

EXECUTORE DIVISO IN 2 NODI
14 LLM

CSPECIFICATION

Il controllo dell'esecuzione resta però all'Executor, che applica i vincoli dello step, gestisce il budget di turni e raccoglie il risultato finale.

Il comportamento durante lo step è regolato da un system prompt dedicato, il cui scopo è verificare il percorso previsto senza aggirare le eventuali anomalie della macchina. Le regole principali sono:

1. **Interazione esclusivamente in-band.** Con il target si interagisce solo attraverso i servizi e i canali esposti dalla challenge. L'host di virtualizzazione non può essere usato per ispezionare, modificare o riavviare la macchina.
2. **Rispetto dell'intended path.** Tutti i requisiti della checklist vanno soddisfatti e un passaggio previsto non può essere sostituito da una scorciatoia equivalente.
3. **Verifica tramite evidenze.** Ogni voce della checklist deve essere associata a un risultato osservato durante l'esecuzione.
4. **Evidenze anche per i fallimenti.** Anche uno step fallito va motivato con un riscontro concreto, come un messaggio di errore o l'esito di un controllo. Se un segnale atteso non compare, non si può concludere subito che lo step sia fallito: prima va verificato con un comando mirato.
5. **Gestione delle sessioni e del terminale.** La shell locale della macchina Kali va distinta dalle sessioni aperte sul target. Prompt di autenticazione e sessioni persistenti vanno gestiti senza chiuderli per errore e, dopo l'uso di applicazioni interattive, il terminale va riportato a uno stato utilizzabile.

Ogni `TestStep` viene eseguito con un numero limitato di turni, per evitare che l'agente ripeta all'infinito tentativi simili. Un limite fisso, però, rischierebbe di interrompere gli step che richiedono più interazioni. Per questo l'Executor assegna un budget iniziale, estendibile tramite il tool `request_turn_extension` fino a un tetto massimo per step. I due valori sono configurabili e di default sono impostati a 8 e 20 turni.

SE A 20
NO

Quando mancano due turni alla fine del budget, e di nuovo all'ultimo turno, viene inviato un avviso con i turni rimanenti: a quel punto l'agente può chiedere un'estensione, se ancora disponibile, oppure chiudere lo step.

Lo step si chiude con la chiamata al tool `submit_step_result`, i cui campi vengono validati dall'Executor e raccolti in un oggetto tipizzato `StepResult`. Lo `StepResult` registra l'identificativo dello step, una sintesi delle operazioni svolte, i valori estratti, i turni usati, le chiamate ai tool e la valutazione della checklist, con l'esito e l'evidenza di ogni voce.

— Lo status dichiarato dall'agente non viene accettato così com'è: il codice lo ricalcola + CONTESTO

a partire dagli esiti della checklist e, se anche una sola voce non è soddisfatta, marca lo step come **FAILED**. Le evidenze restano associate al risultato, ma il codice non ne interpreta il contenuto.

I dati ottenuti durante gli step, come username, password o token, vengono salvati in `verified_values`. All'inizio di ogni step l'agente riceve l'elenco di questi valori: quelli brevi sono mostrati direttamente, mentre per quelli più lunghi, come una chiave privata o un output su più righe, compare solo il nome. Il contenuto si recupera con `get_verified_value`, e l'elenco può essere richiesto di nuovo con `show_verified_values`. In questo modo i dati già raccolti non vengono reinseriti per intero a ogni step e la finestra di contesto resta più contenuta.

3.7 Il Bridge di esecuzione: gestione dei tool e delle interazioni con il target

Le azioni scelte dall'agente vengono eseguite attraverso il Bridge. Il Bridge non è un nodo del grafo, ma un modulo Python usato dall'Executor per accedere ai tool installati sulla macchina Kali. Il suo compito è nascondere all'Executor le differenze tra i backend: una richiesta può essere inoltrata a un tool stateless esposto da HexStrike [14] oppure a una sessione terminale persistente gestita dal Terminal Gateway. Grazie a questa separazione, nuovi tool o backend possono essere aggiunti senza modificare l'Executor.

Non tutti i tool richiedono lo stesso tipo di interazione. Scansioni ed enumerazioni possono essere eseguite come chiamate indipendenti, mentre autenticazioni, shell remote o privilege escalation richiedono che lo stato della sessione venga mantenuto da un turno all'altro. Per questo il Bridge usa due livelli di esecuzione:

- il primo si appoggia a HexStrike per i tool stateless, come `nmap` o `hydra`: il Bridge inoltra la richiesta al server in esecuzione sulla macchina Kali e restituisce il risultato;
- il secondo usa un Terminal Gateway sviluppato appositamente per VulcaTest, che mantiene sessioni PTY⁴ persistenti tramite il modulo Python `pexpect` [17]. Le chiamate successive al tool `interactive_terminal_exec` proseguono sulla stessa shell, e ogni sessione è identificata da un `session_name`, così da poter mantenere più terminali aperti contemporaneamente.

Prima di essere restituiti all'agente, gli output passano dal Bridge. Nelle sessioni PTY vengono rimosse le sequenze di controllo ANSI/VT100 e normalizzati i caratteri

⁴Un PTY (*pseudo-terminal*) è una coppia di dispositivi virtuali che emula un terminale reale: i programmi in esecuzione si comportano come se fossero collegati a una tastiera e a uno schermo, e possono quindi essere controllati da un altro processo.

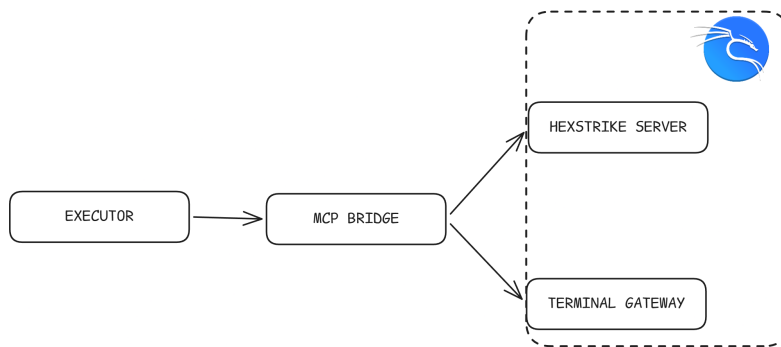


Figura 3: Architettura del Bridge di esecuzione: disaccoppiamento tra tool stateless (HexStrike) e sessioni PTY persistenti (Terminal Gateway).

di fine riga, così da ottenere testo pulito. Vengono anche riconosciuti alcuni stati dell'interazione, come una richiesta di password o di passphrase ancora in attesa, che vengono segnalati insieme all'output. Infine, le descrizioni dei tool vengono aggiornate in base alle sessioni aperte: a ogni `session_name` è associata l'ultima riga prodotta dal terminale, così l'agente sa quali terminali sono attivi e in che stato si trovano, e può scegliere tra una sessione persistente e un tool stateless.

Due meccanismi limitano l'occupazione della finestra di contesto. Gli output molto lunghi vengono troncati oltre una soglia prestabilita, con l'indicazione che l'originale era più lungo. Inoltre, in ogni step sono disponibili solo i tool indicati negli `allowed_tools` del `TestStep`, più alcuni tool sempre presenti per la gestione del terminale (*tool-slicing*). Se il Planner indica "*", vengono resi disponibili tutti.

Le sessioni PTY devono supportare anche applicazioni interattive come `nano`, `vi`, `vim`, `less` o `more`. A differenza dei normali comandi di shell, questi programmi reagiscono direttamente ai tasti premuti. Per gestirle, il Bridge permette di indicare tasti come `enter`, `esc`, `tab`, `backspace`, le frecce direzionali e combinazioni come `ctrl+x` tramite identificatori testuali, che vengono convertiti nelle sequenze corrispondenti e inviati alla sessione. Più input possono essere inviati in sequenza attraverso il parametro `commands`.

Un caso particolare riguarda il tasto Invio. In un terminale, la pressione di Enter invia il carattere Carriage Return (`\r`), non il newline (`\n`). Molte applicazioni interattive distinguono i due caratteri: in `nano`, ad esempio, `\n` corrisponde alla combinazione `Ctrl+J`, che giustifica il testo invece di andare a capo o confermare un comando. Per questo il Bridge traduce `enter` in `\r`, in modo che il tasto produca lo stesso effetto della pressione reale.

3.8 Il Final Evaluator: valutazione finale e Root Cause Analysis

Il Final Evaluator viene eseguito al termine dell'Attack Plan, oppure quando un fallimento interrompe il test, e genera gli artefatti conclusivi della verifica a partire dallo stato della run e dai risultati degli step. Come il Planner, è diviso in due stadi: il primo calcola in modo deterministico i dati della run, il secondo usa un LLM per interpretare le evidenze e, se necessario, individuare la causa del fallimento.

Stadio 1 – Raccolta delle metriche. Il primo stadio è scritto interamente in Python e raccoglie le metriche usate anche nella valutazione sperimentale del Capitolo 5. Da `VulcaTestState`, dagli `StepResult` e dai record delle chiamate ai tool ricava i dati della run: lo stato finale, l'avanzamento del test, i tempi di esecuzione, il numero di chiamate ai tool e i token consumati da ciascuna componente. Questi dati vengono salvati in `run_summary.json`, insieme ai valori verificati e ai risultati dei singoli step, e possono essere elaborati in seguito senza dipendere dalla valutazione dell'LLM.

Stadio 2 – Diagnosi del fallimento e Root Cause Analysis. Il secondo stadio interpreta le evidenze senza modificare l'esito della run. L'LLM riceve l'Attack Plan, gli `StepResult` e gli output rilevanti e produce `REPORT.md`, che riassume lo svolgimento del test e, in caso di fallimento, ne analizza la possibile causa. Se è stata rilevata una non conformità viene generato anche `healing_ticket.json`, che indica lo step che ha bloccato l'esecuzione, la parte della macchina coinvolta, la causa individuata e il tipo di difetto. I tipi previsti sono tre: difetti nella generazione dell'Infrastructure as Code (`IAC_GENERATION_DEFECT`), errori di configurazione (`CONFIG_DEFECT`) ed errori nella specifica del test (`SPECIFICATION_DEFECT`).
1 POTIZZATI

L'ultima categoria serve a distinguere un problema reale della macchina da un criterio di verifica formulato in modo errato. In questo caso la macchina è conforme e non deve essere modificata. Quando invece il difetto riguarda la macchina, il ticket diventa l'input del nodo di healing, descritto nel Capitolo 4.
↳ HUMAN IN THE LOOP

3.9 Prompt engineering e definizione dei ruoli agentici

Ogni componente di VulcaTest che usa un LLM è guidata da un system prompt specifico, scritto in funzione del ruolo che svolge nel workflow.

I prompt sono stati progettati in modo iterativo. Le prime versioni lasciavano più margine di interpretazione e durante i test sono emersi comportamenti come dichiarazioni di successo premature, tentativi di aggirare il percorso previsto o una gestione errata delle sessioni interattive. Questi casi sono stati tradotti, di volta in volta, in vincoli più chiari.

+ FLUIDO

Nel correggere i prompt si è cercato di non specializzarli sui singoli casi incontrati durante lo sviluppo. Una regola costruita attorno a uno scenario specifico rischia di funzionare solo sulle macchine usate nei test, una forma di overfitting del prompt, mentre VulcaTest deve verificare macchine sempre diverse generate da VulcaMind. Per questo ogni nuova istruzione interviene sul comportamento che aveva prodotto l'errore, non sullo scenario in cui si era presentato.

Un esempio riguarda il Planner. Nelle prime versioni separava in fasi distinte operazioni strettamente consecutive. Poiché l'Executor affronta ogni fase con un contesto proprio, all'inizio di una fase poteva trovarsi davanti a un'operazione lasciata incompleta da quella precedente, senza avere le informazioni necessarie per completarla. Il problema è stato risolto descrivendo nel prompt come l'Executor gestisce il contesto tra una fase e l'altra e chiedendo al Planner di accorpare in un'unica fase le operazioni troppo legate tra loro per essere eseguite separatamente.

I prompt non impongono una sequenza prestabilita di azioni: definiscono vincoli e criteri da rispettare, all'interno dei quali l'LLM decide come affrontare il compito. Quando il risultato deve essere usato direttamente dal codice, viene richiesto anche un formato di output prestabilito, come i blocchi YAML dell'Attack Plan, i campi di `submit_step_result` e la struttura di `healing_ticket.json`.

3.10 Modello locale e configurazione dei parametri

BRUTTO

Planner, Executor e Final Evaluator usano un modello eseguito in locale, per i motivi discussi nella Sezione 3.1. La fase di healing usa invece una soluzione diversa, descritta nel Capitolo 4. La configurazione è stata definita in base alle operazioni richieste dal workflow e all'hardware disponibile, intervenendo su quantizzazione, finestra di contesto e parametri di inferenza.

3.10.1 Modello e runtime

Il modello utilizzato è Qwen3.8-27B [18] nella variante quantizzata IQ3_S, distribuita in formato GGUF⁵ nel repository ISTA-DASLab/Qwen3.8-27B-GSQ-RCO-GGUF [19]. Il checkpoint è ottenuto con il metodo di quantizzazione GSQ [20] e con l'ottimizzazione RCO [21]. La quantizzazione riduce l'occupazione in memoria e permette di eseguire un modello da 27 miliardi di parametri sull'hardware disponibile.

Planner ed Executor vengono eseguiti con un livello di reasoning **medium**, che lascia al modello margine sufficiente per analizzare gli output e scegliere le azioni successive senza aumentare troppo i token generati. Il Final Evaluator, che lavora su risultati già raccolti, usa invece un livello **low**.

⁵GGUF è il formato di file usato da llama.cpp per distribuire modelli, anche quantizzati, che raccoglie in un unico file i pesi e i metadati necessari al caricamento.

L'ambiente di sviluppo dispone di una GPU con 16 GB di VRAM e 64 GB di RAM. Eseguire il modello in locale ha richiesto di bilanciare occupazione di memoria, finestra di contesto e velocità di generazione.

Il modello viene eseguito con llama.cpp [22] su ROCm [23], mentre caricamento e configurazione avvengono mediante Unsloth [24], che espone un'API compatibile con quella di OpenAI. La variante IQ3_S usa una quantizzazione a circa 3,5 bit e la KV-cache⁶ è mantenuta a 16 bit. Lo speculative decoding⁷ (MTP e N-gram) aumenta la velocità di generazione, un aspetto importante nelle fasi con molti turni di interazione.

Poiché il workflow non usa in alcun modo la visione, la parte multimodale del modello non viene caricata. Il file `mmproj-Qwen3.8-27B-BF16.gguf`, che contiene il vision encoder e il projector, occupa circa 0,9 GB: escluderlo libera memoria che può essere destinata alla KV-cache, e quindi a una finestra di contesto più ampia.

3.10.2 Dimensionamento della finestra di contesto

+ FLUIDO (La finestra di contesto dipende dalla componente. L'Executor usa una finestra fissa: lavora su contesti contenuti, ma viene invocato molte volte durante il test. Planner e Final Evaluator usano invece un dimensionamento dinamico, perché la quantità di informazioni da elaborare cambia molto da una run all'altra. Tutte queste impostazioni sono gestite da una componente Python dedicata, che le applica prima dell'inferenza senza toccare la logica dei singoli moduli.

Per Planner e Final Evaluator la finestra viene stimata a partire dalla lunghezza dell'input. Il numero di token è approssimato come:

$$T_{\text{input}} = \left\lfloor \frac{C_{\text{tot}}}{k} \right\rfloor + 1 \quad (1)$$

dove C_{tot} è il numero di caratteri dell'input e k il rapporto caratteri/token usato per la stima, di default pari a 3. Il valore è volutamente basso e porta a sovrastimare i token: una finestra un po' più grande del necessario è preferibile a un piano troncato.

A questa stima si aggiunge una riserva R_{out} per l'output del modello, reasoning compreso, e si applica un margine di sicurezza α :

$$T_{\text{richiesti}} = \lfloor (T_{\text{input}} + R_{\text{out}}) \cdot \alpha \rfloor \quad (2)$$

⁶La KV-cache è la memoria in cui il modello conserva le chiavi e i valori del meccanismo di attenzione già calcolati per i token precedenti, così da non doverli ricalcolare a ogni nuovo token. La sua occupazione cresce con la lunghezza del contesto.

⁷Nello speculative decoding alcuni token vengono proposti in anticipo da un meccanismo più veloce e poi verificati dal modello principale in un unico passaggio; i token corretti vengono accettati, riducendo il tempo di generazione senza modificare l'output. Con MTP (*Multi-Token Prediction*) le proposte provengono da teste aggiuntive del modello che predicono più token futuri; con N-gram vengono ricavate da sequenze di token già presenti nel contesto.

↑
~o

Di default $\alpha = 1,15$, mentre R_{out} vale 8192 token per il Planner e 4096 per il Final Evaluator.

La finestra viene poi arrotondata al multiplo successivo di $S = 2048$ token e mantenuta tra un minimo W_{base} e un massimo W_{max} :

$$W_{\text{effettiva}} = \min \left(\max \left(W_{\text{base}}, \left\lceil \frac{T_{\text{richiesti}}}{S} \right\rceil \cdot S \right), W_{\text{max}} \right) \quad (3)$$

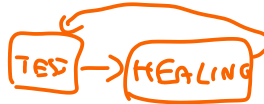
W_{base} , di default 15.872 token, evita finestre troppo piccole quando l’input è breve. W_{max} , di default 40.960 token, è il limite imposto dai 16 GB di VRAM e dal costo in memoria della KV-cache.

Se la finestra necessaria supera W_{max} , la condizione viene segnalata. Nel caso del Planner, un piano troncato viene comunque intercettato dal parser e rifiutato prima dell’esecuzione.

4 Il self-healing closed-loop in VulcaTest

VulcaTest permette di rilevare una non conformità e di raccogliere le evidenze che ne descrivono la causa. Il solo rilevamento, però, lascia ancora all'operatore il compito di intervenire sui file sorgente della macchina e di verificare nuovamente il risultato.

Per chiudere questo ciclo VulcaTest include un nodo di healing, che parte dalla diagnosi del Final Evaluator, corregge i file Infrastructure as Code della macchina, la ricostruisce e avvia un nuovo test.



4.1 Il nodo di healing nel workflow closed-loop

Il nodo di healing aggiunge al workflow una fase di correzione, successiva al rilevamento di una non conformità. Test e correzione restano due attività separate: affidarle allo stesso agente mescolerebbe compiti che richiedono accessi all'ambiente e punti di vista diversi.

- L'Executor lavora **in-band**: parte dalla macchina Kali e interagisce soltanto con i servizi esposti dal target. Non accede ai file usati per generare la macchina, perché queste informazioni non sarebbero disponibili a un utente che affronta la challenge.
- Il nodo di healing lavora invece **out-of-band** sui file sorgente dell'infrastruttura, cioè sui file da cui la macchina viene generata: ricette, playbook Ansible, Dockerfile e sorgenti applicativi.

La separazione mantiene anche indipendenti verifica e correzione. Il nodo di healing può modificare la macchina, ma non i criteri con cui VulcaTest stabilisce la conformità. L'Executor può verificare la macchina, ma non correggerla durante il test.

Il nodo `healer` completa lo `StateGraph` descritto nella Sezione 3.5. Viene attivato dopo il Final Evaluator se la run non è conforme, se il self-healing è abilitato e se restano tentativi di correzione disponibili. Dopo l'intervento il controllo torna al nodo `orchestrator` e il test riparte con lo stesso Attack Plan.

4.2 Il ticket di healing

La diagnosi prodotta dal Final Evaluator arriva al nodo di healing sotto forma di ticket, un file JSON generato quando la run non è conforme (Sezione 3.8). Il ticket indica lo step in cui il test si è bloccato, il tipo di difetto, la parte della macchina coinvolta, una descrizione del problema e una correzione suggerita (*recommended patch*).

Il controller Python che gestisce il nodo di healing non inserisce il contenuto del ticket nel prompt, ma indica all'agente di healing i percorsi dei file da consultare, tra cui il ticket e il report del test. È l'agente ad aprirli e a decidere quali altri file esaminare.

Il ticket va letto tenendo conto di come è stato prodotto. Il Final Evaluator conosce soltanto le evidenze raccolte dall'Executor, cioè ciò che si osserva dall'esterno della macchina, e anche la correzione suggerita nasce da questa prospettiva. Il punto in cui l'errore viene rilevato può quindi non coincidere con l'origine del difetto. Una risposta HTTP errata o un servizio non disponibile, ad esempio, possono dipendere da un task di provisioning non eseguito o da una configurazione generata in modo sbagliato.

4.3 L'agente di healing: delega a un harness agentico

La fase di healing richiede capacità diverse da quelle del conformance testing. L'Executor interagisce con un target già costruito, entro i vincoli definiti dal `TestStep`, dalla checklist e dai tool disponibili. L'agente di healing deve invece navigare i sorgenti del progetto, mettere in relazione file differenti e applicare modifiche di natura molto varia. Può trattarsi di correggere un task Ansible o un Dockerfile, ma anche di generare file mancanti, come una pagina PHP o uno script richiesto dalla challenge.

LAFABDIAMARCO

Nel collaudo un harness general-purpose è stato scartato perché lascia all'agente troppa libertà (Sezione 3.1). Nella correzione questa libertà diventa utile. Non si può stabilire in anticipo quali file andranno modificati o quali strumenti serviranno, e un harness generico mette a disposizione dell'agente gli strumenti per esplorare il repository e intervenire dove serve.

Per questo la modifica dei sorgenti è affidata a Google Antigravity [11], con il modello Gemini 3.8 Flash e reasoning impostato su High. Anche questo nodo potrebbe usare un modello locale, ma i modelli eseguibili sull'hardware disponibile (Sezione 3.10.1) hanno capacità limitate nella scrittura e correzione di codice, mentre i modelli di frontiera sono ottimizzati proprio per questo tipo di compito.

Il nodo `healer` avvia la CLI di Antigravity (`agy`) come sottoprocesso Python, in modalità non interattiva. L'obiettivo della correzione viene passato nel prompt con il comando `/goal`, una funzione di Antigravity che permette all'agente di perseguire un obiettivo in autonomia. L'agente è configurato per applicare le modifiche direttamente sui file e per non chiedere conferme sui permessi, così la sessione prosegue senza l'intervento di un operatore. Gli eventi della sessione arrivano in tempo reale in formato JSON e vengono registrati, compresi i token consumati e i tool usati.

La libertà dell'agente resta comunque delimitata dalle regole del prompt (Sezione 4.4) e dai controlli che il codice applica alle modifiche (Sezione 4.5). La dipendenza da Antigravity, inoltre, è confinata a questo livello: le fasi successive non dipendono dall'harness usato.

4.4 Prompt dell'agente di healing e perimetro di modifica

Come per le componenti descritte nella Sezione 3.9, anche il comportamento dell'agente di healing è delimitato da un prompt dedicato. Rispetto a quello dell'Executor, il prompt è meno rigido. L'agente non segue un piano diviso in step con una checklist da soddisfare, perché la correzione da applicare non è nota in anticipo e può richiedere interventi molto diversi tra loro. Il prompt non descrive come correggere la macchina, ma fissa le regole entro cui l'agente può muoversi.

Il prompt impone all'agente cinque vincoli:

1. **Modifica minima.** La correzione interessa soltanto ciò che serve a risolvere la non conformità.
2. **Nessun leakage didattico.** La modifica non deve introdurre suggerimenti, credenziali in chiaro o altre informazioni che semplifichino la challenge.
3. **Preservazione delle vulnerabilità previste.** Ciò che un agente di coding tratterebbe come un problema di sicurezza, ad esempio una SQL injection, un binario SUID o una configurazione `sudo` debole, può far parte del percorso didattico e non va corretto.
4. **Nessuna modifica arbitraria.** Se l'analisi non individua un difetto nei sorgenti, l'agente termina senza applicare una patch.
5. **Esplorazione limitata alla macchina interessata.** Ricerche e modifiche restano sui file della challenge corrente.

Oltre ai vincoli, il prompt definisce il perimetro di lettura e scrittura. L'agente può leggere l'intero workspace, perché per risalire all'origine del difetto deve confrontare documentazione, report del test e sorgenti. Può invece scrivere soltanto sui sorgenti della macchina corrente, cioè la ricetta da cui VulcaForge la genera e gli eventuali sorgenti applicativi dedicati.

La documentazione della challenge e il report di VulcaTest restano in sola lettura. Lo stesso vale per i file che VulcaForge genera dalla ricetta, come il Dockerfile e il playbook Ansible: vengono ricreati a ogni generazione e una modifica applicata lì andrebbe persa. Il codice di VulcaTest, le altre challenge e l'ambiente host non possono essere modificati.

Ogni tentativo di correzione avvia una nuova sessione dell'agente, che non conserva il contesto dei tentativi precedenti. Le informazioni da trasmettere da un tentativo al successivo vengono salvate sul filesystem e indicate nel prompt.

Il feedback è di due tipi. Se la modifica impedisce la compilazione, per un errore nel build Docker o nell'esecuzione del playbook Ansible, la macchina non viene avviata e l'errore viene salvato in `BUILD_ERROR.md`. Al tentativo successivo il contenuto del file viene inserito all'inizio del prompt, e l'agente deve rendere di nuovo compilabile la macchina prima di occuparsi della non conformità.

Se invece la compilazione riesce ma il nuovo test fallisce, l'agente riceve il report aggiornato, che descrive il comportamento della macchina dopo la modifica.

4.5 Tracciamento e validazione delle modifiche

Come per gli step del test, anche nella fase di healing ciò che l'agente dichiara di aver fatto non viene preso come prova del risultato. Il controller confronta lo stato dei file prima e dopo la sessione e salva le modifiche effettuate come evidenza.

Il tracciamento delle modifiche è implementato in un modulo Python. Prima dell'avvio dell'agente viene registrato lo stato dei file del repository di VulcaForge e viene salvata una copia dei sorgenti della macchina, usata come riferimento e per eventuali ripristini.

Al termine della sessione il filesystem viene confrontato con lo snapshot iniziale. Dal confronto vengono prodotti `patch.diff`, che contiene le differenze riga per riga, e `HEALING_REPORT.md`, che elenca i file modificati. Il diff permette anche di misurare l'ampiezza della patch, una delle metriche usate nel Capitolo 5.

Lo stesso confronto mostra se l'agente ha scritto fuori dal perimetro definito nella Sezione 4.4. In questo caso le modifiche esterne vengono annullate ripristinando lo stato dello snapshot, prima che la macchina venga ricostruita. I file coinvolti vengono elencati in `PERIMETER_VIOLATIONS.md`, così il tentativo resta documentato anche se i suoi effetti sono stati annullati.

Il codice di uscita dell'agente da solo non basta a capire se sia stata applicata una correzione. Per questo l'esito della sessione viene classificato in base alle modifiche rilevate sul filesystem e all'esito dell'agente:

- **PATCHED**: almeno una modifica valida all'interno del perimetro.
- **DECLINED**: la sessione termina senza errori, ma senza modifiche ai sorgenti autorizzati.
- **OUT_OF_SCOPE**: tutte le modifiche rilevate erano fuori dal perimetro e sono state annullate.
- **ERROR**: l'harness termina con un errore o supera il tempo massimo.

NON ERA
OBBLIGO
A NON
FARLO?

La classificazione determina anche il passo successivo. Solo nel caso `PATCHED` la macchina viene ricostruita e testata di nuovo. Negli altri casi non c'è nessuna modifica da verificare e il workflow termina.

4.6 Chiusura del ciclo: rebuild dell'ambiente e regression testing

Una modifica ai sorgenti non basta a considerare risolta la non conformità. La macchina deve essere ricostruita a partire dai sorgenti corretti e deve superare un nuovo test.

Dopo la modifica, il nodo di healing richiama il generatore di VulcaForge, che traduce la ricetta aggiornata in un nuovo Dockerfile e in un nuovo playbook Ansible. L'immagine viene poi compilata sulla macchina Kali attraverso il Terminal Gateway, e il controller considera concluso il build solo quando nell'output compare un messaggio di conferma stampato al termine della compilazione. In questo modo non serve fissare un timeout: il controller attende il tempo effettivamente richiesto dalla compilazione, che varia da una macchina all'altra.

Se la compilazione fallisce, l'errore viene salvato in `BUILD_ERROR.md` (Sezione 4.4) e il nuovo test non viene avviato.

Se il build riesce, il container precedente viene sostituito da uno nuovo, avviato dall'immagine appena compilata. Il suo indirizzo IP viene recuperato e salvato nello stato condiviso come nuovo target.

Il test non riparte dallo step fallito, ma dall'inizio dell'Attack Plan. Una modifica che risolve lo step fallito potrebbe aver alterato un passaggio precedente, per questo l'esito della correzione viene valutato sull'intera macchina, con gli stessi criteri del test iniziale.

Nella ricostruzione può anche verificarsi un guasto degli strumenti di test, come una disconnessione del Terminal Gateway o il mancato recupero dell'indirizzo IP del container. Questo caso viene tenuto distinto da un errore di compilazione: il guasto è registrato nello stato come errore dell'infrastruttura e riportato in `run_summary.json`. Senza questa distinzione, un nuovo test lanciato contro un container non raggiungibile verrebbe registrato come una non conformità della macchina, mentre il problema riguarda lo strumento di test.

Il numero di interventi del nodo di healing è limitato da un parametro configurabile. Senza questo limite, una correzione che non risolve il problema potrebbe innescare un ciclo senza fine di modifiche, ricostruzioni e nuovi test. Raggiunto il limite, il workflow termina con l'esito dell'ultimo test.

Con questa fase il ciclo si chiude: una non conformità può essere diagnosticata, corretta sui sorgenti e verificata di nuovo con gli stessi criteri. Il Capitolo 5 valuta separatamente le prestazioni del test, della diagnosi e della correzione.

5 Valutazione sperimentale

- 5.1 Obiettivi della valutazione e Test Oracle Problem
- 5.2 Metriche di valutazione
- 5.3 Dataset: macchine di riferimento e perturbazioni controllate
- 5.4 Configurazione sperimentale
- 5.5 Risultati
 - 5.5.1 Riconoscimento, diagnosi e riparazione
 - 5.5.2 Confronto tra modelli
 - 5.5.3 Studio di ablazione dei ruoli
- 5.6 Casi di studio
 - 5.6.1 Pizzeria_B2R: validazione closed-loop end-to-end
- 5.7 Discussione e limiti della valutazione

6 Conclusioni e sviluppi futuri

6.1 Sintesi dei contributi

6.2 Limiti della soluzione attuale

6.3 Sviluppi futuri

Riferimenti bibliografici

- [1] Cyber Range Project Team, NICE Community Coordinating Council. The Cyber Range: A guide. National Institute of Standards and Technology (NIST), https://nist.gov/system/files/documents/2023/09/29/The%20Cyber%20Range_A%20Guide.pdf, 2023. Accessed: 2026-10-04.
- [2] Muhammad Mudassar Yamin, Basel Katt, and Vasileios Gkioulos. Cyber ranges and security testbeds: Scenarios, functions, tools and architecture. *Computers & Security*, 88:101636, 2020. doi: 10.1016/j.cose.2019.101636.
- [3] Red Hat, Inc. Ansible. <https://docs.ansible.com/>, 2026. Accessed: 2026-10-04.
- [4] Docker, Inc. Docker. <https://docs.docker.com/>, 2026. Accessed: 2026-10-04.
- [5] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- [6] Lei Wang, Chengbang Ma, Xueyang Feng, Zeyu Zhang, Hao-ran Yang, Jingsen Zhang, Zhi-Yang Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.
- [7] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peethawatthachai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg,

- Derek Askaryar, Haoxiang Yang, Aolin Zhang, Rishi Alluri, Nathan Tran, Rinara Sangpisit, Kenny Oseleononmen, Dan Boneh, Daniel Ho, and Percy Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [11] Google. Google antigravity documentation: Agent. <https://www.antigravity.google/docs/agent/>, 2026. Documentazione ufficiale. Accesso: 3 ottobre 2026.
 - [12] Anthropic. Claude Code documentation. <https://code.claude.com/docs>, 2026. Accessed: 2026-10-04.
 - [13] Model Context Protocol. Model Context Protocol specification (version 2025-06-18). <https://modelcontextprotocol.io/specification/2025-06-18>, 2025. Accessed: 2026-10-04.
 - [14] OTT Cybersecurity LLC. HexStrike AI MCP Agents (Version 6.0). <https://github.com/0x4m4/hexstrike-ai>, 2026. Repository GitHub ufficiale gestito da 0x4m4. Accesso: 3 ottobre 2026.
 - [15] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864. USENIX Association, 2024.
 - [16] LangChain, Inc. LangGraph. <https://github.com/langchain-ai/langgraph>, 2026. Accessed: 2026-10-03.
 - [17] Noah Spurrier and Pexpect contributors. Pexpect: A pure Python module for spawning child applications (version 4.9.0). <https://pexpect.readthedocs.io/>, 2023. Release PyPI 4.9.0 (25 novembre 2023). Accesso: 3 ottobre 2026.
 - [18] Qwen Team. Qwen3.8-max: A new bar for coding and cowork. <https://qwen.ai/blog?id=qwen3.8>, 2026. Accessed: 2026-10-03.
 - [19] ISTA-DASLab. Qwen3.8-27B-GSQ-RCO-GGUF. <https://huggingface.co/ISTA-DASLab/Qwen3.8-27B-GSQ-RCO-GGUF>, 2026. Accessed: 2026-10-03.
 - [20] Alireza Dadgarnia, Soroush Tabesh, Mahdi Nikdan, Michael Helcig, Eldar Kurtic, and Dan Alistarh. GSQ: Highly-accurate low-precision scalar quantization for LLMs via Gumbel-Softmax sampling. *arXiv preprint arXiv:2604.18556*, 2026.
 - [21] Michael Helcig and Dan Alistarh. Model compression with exact budget constraints via Riemannian manifolds. *arXiv preprint arXiv:2605.00649*, 2026.

- [22] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2026. Repository GitHub ufficiale. Accesso: 3 ottobre 2026.
- [23] Advanced Micro Devices, Inc. AMD ROCm software ecosystem documentation. <https://rocm.docs.amd.com/>, 2026. Documentazione ufficiale. Accesso: 3 ottobre 2026.
- [24] Daniel Han, Michael Han, and Unsloth Team. Unsloth: Fast and memory efficient LLM fine-tuning and inference. <https://github.com/unslothai/unsloth>, 2023. Accessed: 2026-10-03.

Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.